

Universidad Nacional Autónoma de México

Facultad de Ciencias

Reporte 01

Esquemas de Codificación

No. de cuenta	Nombre
320148204	González Lucero Alan Uriel
321225087	Rivera Jiménez Isaac
321306102	Tapia Sánchez Jesús



Complejidad Computacional

Semestre 2027-1

Índice

1 Problema **2**

2 Ejercicio 1 **2**
2.1 Solución 2

3 Ejercicio 2 **3**
3.1 Solución 3

4 Ejercicio 3 **6**
4.1 Solución 6

5 Ejercicio 4 **11**
5.1 Solución 11

6 Referencias Bibliográficas **12**

Problema

Enunciado

"DeliCiencias" es un nuevo proyecto estudiantil de logística y entrega de comida a domicilio que busca competir optimizando las rutas de sus repartidores. **Cada repartidor** utiliza una mochila térmica de alta tecnología que tiene una **capacidad máxima de peso** estricta. En un día de alta demanda, el sistema central recibe decenas de pedidos simultáneos de distintos locales aliados (pizzas, ensaladas saludables, hamburguesas, etc.). **Cada pedido pendiente** i tiene un **peso físico** w_i (en gramos) y **representa una ganancia** económica v_i (en pesos) para el repartidor.
El objetivo es diseñar un esquema para representar el catálogo de pedidos pendientes en un archivo, y proponer un algoritmo que determine si un repartidor puede seleccionar un subconjunto de estos pedidos tal que quepan en su mochila térmica sin superar el límite de peso W, y que en conjunto le garanticen una ganancia de al menos V pesos.

Ejercicio 1

Enunciado

(a) ¿Cómo podemos expresar este problema en su versión de búsqueda?

(b) ¿Tiene sentido plantearse una versión de optimización?

(c) Plantea el problema en su versión de decisión, y enúncialo en su forma canónica.

Solución

(a) **Un problema de búsqueda** consiste es encontrar y devolver la estructura exacta que cumple con las condiciones establecidas.

Para el caso de "DeliCiencias", la versión de búsqueda se expresaría formalmente de la siguiente manera:

Dado un catálogo de pedidos pendientes donde cada pedido i tiene un peso físico w_i y una ganancia económica v_i, una capacidad máxima de mochila W y una ganancia meta V, **encuentra y devuelve** un subconjunto específico de pedidos S tal que la suma de sus pesos no exceda W ($\sum_{i \in S} w_i \leq W$) y la suma de sus ganancias alcance o supere la meta V ($\sum_{i \in S} v_i \geq V$). Si no existe ninguna combinación que cumpla ambas condiciones, el algoritmo debe reportarlo.

(b) Si, tiene sentido, ya que se busca maximizar la ganancia del repartidor aprovechando al máximo la capacidad W de la mochila, sin depender de un umbral fijo V.

La versión de optimización modificaría la estructura del problema al eliminar el parámetro de entrada V. El algoritmo ya no se limitaría a buscar una combinación que

simplemente cruce un umbral, sino que evaluaría el catálogo para encontrar el subconjunto exacto de pedidos cuya suma de pesos no exceda W y cuya suma de ganancias sea el máximo absoluto posible.

(c) **OPTIMIZACIÓN DE ENTREGAS - VERSIÓN DE DESICIÓN.**
Datos (parámetros): Un conjunto finito de pedidos pendientes $P = \{p_1, p_2, \ldots, p_n\}$. Para cada pedido $p_i \in P$, un peso físico $w_i \in \mathbb{Z}^+$ y una ganancia económica $v_i \in \mathbb{Z}^+$. Una capacidad máxima de la mochila $W \in \mathbb{Z}^+$ y una ganancia meta $V \in \mathbb{Z}^+$.

Pregunta (objetivo): ¿Existe un subconjunto $S \subseteq P$ tal que la suma de los pesos de los pedidos en S sea menor o igual a W ($\sum_{p_i \in S} w_i \leq W$) y la suma de sus ganancias sea mayor o igual a V ($\sum_{p_i \in S} v_i \geq V$)?

Ejercicio 2

Enunciado

(a) Describe un esquema de codificación para ejemplares del problema anterior (en su versión de decisión).
Nota: No es válido usar bibliotecas o herramientas externas que ya lean formatos predefinidos (como JSON, XML, CSV, etc).

(b) Describe e implementa un algoritmo que lea archivos con el formato propuesto en el inciso anterior e imprima en consola los datos del ejemplar.
El programa debe recibir como entrada (desde consola):

- Nombre del archivo de entrada con el ejemplar a leer.
  El ejemplar debe seguir el esquema de codificación propuesto

Como resultado de la ejecución, el programa deberá producir como salida:

- Imprimir en consola el total de pedidos disponibles
- Imprimir en consola la capacidad máxima de la mochila y la ganancia meta
- Imprimir dos ejemplos de pedidos disponibles, mostrando su peso y ganancia

Solución

(a) Ya que el diseño de un esquema de codificación consiste en establecer reglas exactas para traducir los parámetros de un ejemplar a una cadena de texto que un programa pueda procesar, y debido a que en la práctica se nos prohíbe utilizar formatos estructurados ya establecidos, vamos a crear un formato posicional basado en saltos de línea y un delimitador simple (como el espacio).

Así que, para la versión de decisión, necesitamos almacenar el total de pedidos (n), la capacidad máxima (W), la ganancia meta (V), y las características de cada pedido (w_i y v_i).

Obteniendo la siguiente estructura para el archivo de texto plano:

- Línea 1: Un número entero n que define el total de pedidos disponibles.
- Línea 2: Un número entero W que defina la capacidad máxima de la mochila (en gramos).
- Línea 3: Un númeroo entero V que define la ganancia meta (en pesos).
- Línea 4 a la $n+3$: Dos números enteros separados por un espacio, correspondientes a w_i (pesos) y v_i (ganancias) del pedido i.

(b) **1. Descripción del Algoritmo**

Entrada: Leer desde los argumentos de la consola el nombre del archivo de texto.

Lectura: Abrir el archivo en modo lectura.

Decodificación (Parseo):

- Leer la línea 1 y asignarla a n (total de pedidos).
- Leer la línea 2 y asignarla a W (capacidad de la mochila).
- Leer la línea 3 y asignarla a V (ganancia meta).
- Crear una lista iterando desde la línea 4 en adelante. Por cada línea, separar los valores por el espacio en blanco para obtener el peso (w_i) y la ganancia (v_i).

Salida: Imprimir n, W, y V. Luego, acceder a los dos primeros elementos de la lista de pedidos pendientes e imprimir sus datos.

2. Implementación en Python

La implementación del algoritmo se encuentra en el archivo `lector.py`

3. Análisis del Segundo Algoritmo Propuesto de Bajo Consumo

(a) **Esquema de codificación binaria de longitud fija.** Para codificar las instancias de este problema se usa un **esquema de codificación en binario puro** ($\Sigma = \{0, 1\}$) con **longitud fija de 16 bits** (2 bytes) por entero:

- **Dominio y Rango:** Cada número entero no negativo ($x \in \mathbb{Z}_{\geq 0}$) se convierte a su representación binaria de 16 bits con relleno de ceros a la izquierda (*zero-padding*), lo que permite representar valores en el rango $[0, 65\,535]$.

- **Estructura de la Cadena Binaria:** La cadena $C \in \{0,1\}^*$ se compone de n bloques de 32 bits para los n pedidos, seguidos por 2 bloqes finales de 16 bits para los parámetros globales W y V:

$$C = \underbrace{\text{bin}_{16}(w_1)\,\text{bin}_{16}(v_1)\ldots\text{bin}_{16}(w_n)\,\text{bin}_{16}(v_n)}_{n \text{ bloques de 32 bits para pedidos}}\ \underbrace{\text{bin}_{16}(W)}_{16 \text{ bits}}\ \underbrace{\text{bin}_{16}(V)}_{16 \text{ bits}}$$

- **Decodificación Unívoca:** Debido a que la longitud total del archivo satisface $|C| = 32n + 32$, los últimos 32 bits corresponden a la capacidad máxima W (bits $|C| - 32$ a $|C| - 16$) y a la ganancia meta V (bits $|C| - 16$ a $|C|$). Los bits anteriores se procesan en parejas de 16 bits para reconstruir cada tupla de pedido (w_i, v_i).

(b) **Descripción del Algoritmo e Implementación.** El algoritmo lee la cadena binaria almacenada en el archivo especificado, extrae los datos mediante cortes exactos de bits (*slicing*) y despliega la información requerida. **Pasos del Algoritmo:**

1. Solicitar por consola el nombre del archivo de entrada y abrirlo en modo lectura de texto.

2. Separar la cadena C: los últimos 16 bits se convierten a entero base 2 ara obtener V, los 16 bits precedentes para W, y el subsegmento anterior $C[0 : |C| - 32]$ representa el conjunto de pedidos.

3. Iterar sobre el subsegmento en saltos de 32 bits, convirtiendo los primeros 16 bits a peso w_i y los siguientes 16 bits a ganancia v_i.

4. Imprimir en consola: el número total de pedidos n, la lista completa de los pedidos, la capacidad W, la ganancia meta V y dos ejemplos representativos de los pedidos con información más detallada.

(c) **Implementación en Python.** La implementación del algoritmo se encuentra en el archivo **ejercicio2.py**

Ejercicio 3

Enunciado

Supongamos que, debido a una promoción agresiva de la aplicación, todos los pedidos disponibles en el sistema tienen exactamente el mismo peso (por ejemplo, todos pesan 500 gramos), aunque la ganancia de cada uno sigue siendo diferente dependiendo de la distancia de entrega.
Describe e implementa un algoritmo eficiente para resolver el problema planteado, para este caso particular (pesos idénticos). La implementación del algoritmo debe hacer uso del esquema de codificación implementado en el inciso 2.
El programa implementado debe recibir como entrada (desde consola) el nombre del archivo que contiene el ejemplar a probar, y como salida deberá imprimir:

- Respuesta al problema de decisión planteado (SÍ o NO se puede alcanzar la ganancia meta sin romper la mochila).
- Salida adicional: Si la respuesta es SÍ, imprimir el peso total.

Incluir al menos 2 archivos de prueba (ejemplos de ejecución) para ambos programas (incisos 2 y 3), así como ejemplos visuales de los ejemplares con los que prueben su programa.
Al menos deben agregar ejemplos de los siguientes casos

- ejemplar en el que la respuesta sea un sí
- ejemplar en el que la respuesta sea un no

* En caso de que no se pueda construir alguno de los ejemplares, justifica detalladamente por qué y propón algún otro caso de prueba.

Solución

1. Análisis

Dado que cada pedido pesa exactamente lo mismo (llamémosle w_{const}), el límite de la mochila W ya no restringe combinaciones complejas de pesos, sino simplemente la **cantidad máxima de pedidos** que se pueden llevar.

La cantidad máxima de paquetes k que caben en la mochila se calcula con una división entera:

$$k = \lfloor \frac{W}{w_{const}} \rfloor$$

Al saber que solo podemos llevar a lo mucho k pedidos y que todos consumen exactamente el mismo espacio, la estrategia óptima es seleccionar los k pedidos que ofrezcan la mayor ganancia individual.

Diseño del Algoritmo Eficiente

Lectura de Datos: Se lee el archivo utilizando el esquema que diseñamos en el Ejercicio 2. Se asume que todos los pedidos tienen el mismo peso, por lo que tomamos ese valor como w_{const}.

Cálculo de Límite de Artículos: Se determina cuántos pedidos caben calculando $k = \lfloor W/w_{const} \rfloor$. Si resulta que k es mayor que el número total de pedidos disponibles n, ajustamos $k = n$ (el repartidor tiene espacio para llevar todo el catálogo).

Ordenamiento (Sorting): Se ordena la lista de ganancias de los pedidos en orden descendente (del más lucrativo al menos lucrativo).

Selección: Se toman los primeros k elementos de esta lista ordenada y se suman para obtener la ganancia máxima posible (V_{max}).

Evaluación Final:

- Si $V_{max} \geq V$, el algoritmo imprime "SÍ" y reporta el peso total transportado, el cual será exactamente la cantidad de artículos seleccionados multiplicada por w_{const}.
- Si $V_{max} < V$, el algoritmo imprime "NO".

Ventaja de Complejidad En este caso donde los pesos son idénticos, la complejidad de nuestro algoritmo está dominada únicamente por el paso de ordenamiento de las ganancias, lo que nos da un tiempo de ejecución de $O(n \log n)$. Por lo que se puede considerar un algoritmo "eficiente".

2. Implementación en Python

La implementación del algoritmo se encuentra en el archivo `mochila_pesos_identicos.py`

3. Descripción de los Ejemplos Visuales de los Ejemplares

Ejemplar 1: Caso Afirmativo (Respuesta "SÍ")

Para este caso, diseñaremos un catálogo donde el repartidor tiene espacio limitado, pero al aplicar nuestro algoritmo, los paquetes más rentables logran superar la ganancia meta.

- **Total de pedidos (n):** 4
- **Capacidad de la mochila (W):** 1200 gramos
- **Ganancia meta (V):** 150 pesos

Representación visual (sin codificar):

Pedido	Peso	Ganancia
1	500g	30 pesos
2	500g	100 pesos
3	500g	50 pesos
4	500g	80 pesos

Cadena que codifica al ejemplar (`ejemplar_si.txt`):

```
4
1200
150
500 100
500 80
500 50
500 30
```

Resultado de la ejecución:
Al ejecutar el algoritmo calculará que en la mochila solo caben 2 pedidos enteros (1200 divido entre 500). Tomará las ganancias mayores (100 y 80), sumando 180 pesos. Como 180 es mayor a la meta de 150 pesos, la salida en consola será:

```
SÍ
1000
```

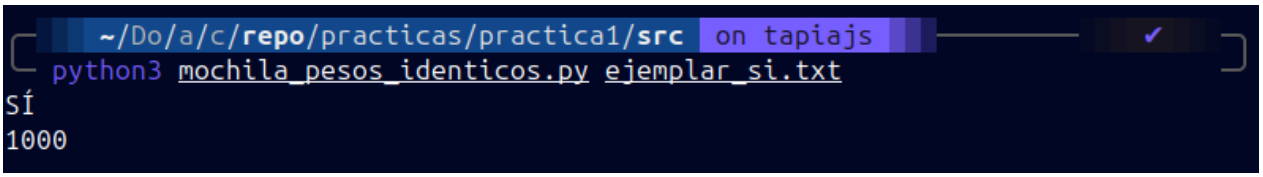


Figura 1: Captura de pantalla de la salida en terminal al ejecutar el algoritmo `mochila_pesos_identicos.py` con el `ejemplar_si.txt`

Ejemplar 2: Caso Negativo (Respuesta "NO")

Aquí plantearemos un escenario restrictivo. La mochila es pequeña y, aunque el repartidor tome el pedido más caro, no será suficiente para alcanzar su meta.

- **Total de pedidos (n):** 3
- **Capacidad de la mochila (W):** 800 gramos
- **Ganancia meta (V):** 120 pesos

Representación visual (sin codificar):

Pedido	Peso	Ganancia
1	500g	40 pesos
2	500g	90 pesos
3	500g	70 pesos

Cadena que codifica al ejemplar (`ejemplar_no.txt`):

```
3
800
120
500 90
500 70
500 40
```

Resultado de la ejecución:
La mochila solo puede transportar 1 pedido (800 dividido entre 500). El mejor pedido disponible ofrece 90 pesos. Dado que 90 es menor que la meta de 120 pesos, es imposible cumplir el objetivo. La salida en consola será:

```
NO
```

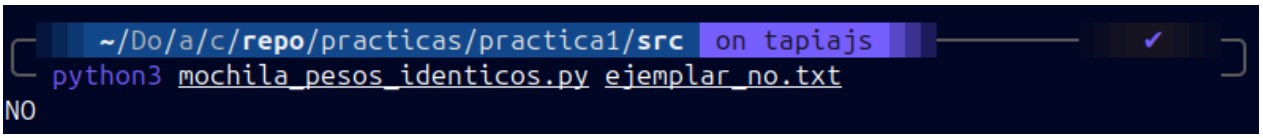


Figura 2: Captura de pantalla de la salida en terminal al ejecutar el algoritmo `mochila_pesos_identicos.py` con el `ejemplar_si.txt`

4. Análisis del Segundo Algoritmo Propuesto de Bajo Consumo

Se sabe que los pesos para cada pedido p son de igual cantidad; de esta manera $p_{w1} = p_{w2} = p_{w3}$.., por lo tanto, lo único que posee un valor independiente p_v son los valores para cada p. Teniendo esto en cuenta, se podría generar una suma de las cantidades de p_v hasta que se alcance un V de ganancias, siempre y cuando no sobrepase W. Esto se puede lograr gracias a un $max - heap$, ya que podemos ordenar los valores de los pedidos dependiendo de la cantidad de ganancia que se pueda ir generando. Para esto se utilizó la biblioteca *heapq* de Python, que ayuda a poder utilizar los montículos mínimos. [1]

Una vez implementados los montículos para las tuplas de pedidos, se convirtieron los valores de ganancias de cada pedido en valores negativos, para que el montículo mínimo tomara los valores negativos como los mejores valores; después se transformaba a un entero positivo con la función *abs*() que convierte en valor absoluto.

La complejidad en tiempo está dada por $O(nlog(m))$, donde

La decodificación y validación de pesos del ejercicio anterior es: $O(n)$, al recorrer la lista de pedidos una vez para verificar que todos tengan el mismo peso w. Construcción del Max-Heap $(heapify) : O(n)$, tiempo lineal para organizar los n elementos en la estructura del montículo. Selección de los mejores elementos: $O(m \log n)$, ya que el bucle realiza como máximo m extracciones, y cada reordenamiento en un montículo de tamaño n toma $O(\log n)$. De esta manera, $O(m \log n)$.

La complejidad en espacio del método es $O(n)$, donde n es la cantidad total de pedidos. Lista de pedidos: Almacena las n tuplas (peso, ganancia) procesadas por desbinarizar $(O(n))$. Estructura max-heap: Crea una nueva lista con n tuplas $(-ganancia, peso)$ para la cola de prioridad $(O(n))$. Variables auxiliares: Variables como maxsum, maxweight, W y V ocupan un espacio constante e independiente del tamaño de la entrada $(O(1))$. Así $O(n)$.

El ejemplar que se toman en cuenta, para el si tiene los datos $ejemplar3$:

```
    Nombre del archivo a guardar (ej. ejemplar.txt): ejemplar3
¿Cuántos pedidos deseas ingresar? 6
Pedido 1:
Peso (en gramos): 300
Ganancia (en pesos): 40
Pedido 2:
Peso (en gramos): 300
Ganancia (en pesos): 24
Pedido 3:
Peso (en gramos): 300
Ganancia (en pesos): 17
Pedido 4:
Peso (en gramos): 300
Ganancia (en pesos): 26
Pedido 5:
Peso (en gramos): 300
Ganancia (en pesos): 8
Pedido 6:
Peso (en gramos): 300
Ganancia (en pesos): 10
Capacidad máxima de la mochila W (en gramos): 900
Ganancia mínima requerida V (en pesos): 75

Cadena binaria generada:

000000010010110000000000001010000000000100101100000000000000110000000000
100101100000000000001000100000001001011000000000000
110100000000100101100000000000000100000000010010110000000000000
101000000011100001000000000001001011
```

El ejemplar que se toman en cuenta, para el si tiene los datos *ejemplar*2:

```
    Nombre del archivo a guardar (ej. ejemplar.txt): ejemplar2
¿Cuántos pedidos deseas ingresar? 4
Pedido 1:
Peso (en gramos): 300
Ganancia (en pesos): 40
Pedido 2:
Peso (en gramos): 200
Ganancia (en pesos): 30
Pedido 3:
Peso (en gramos): 350
Ganancia (en pesos): 17
Pedido 4:
Peso (en gramos): 299
Ganancia (en pesos): 19.5
[Error] Entrada inválida. Por favor, ingresa un número entero.
Ganancia (en pesos): 19
Capacidad máxima de la mochila W (en gramos): 800
Ganancia mínima requerida V (en pesos): 88

Cadena binaria generada:
00000001001011000000000000
10100000000000110010000000000000
11110000000010101111000000000001000
10000000100101011000000000010011
00000011001000000000000001011000
```

Ejercicio 4

Enunciado

Para que las terminales móviles de los repartidores consuman la menor cantidad de datos posible, el protocolo de red requiere un esquema de codificación estrictamente binario. El archivo con la información del ejemplar solo puede contener caracteres de texto ’0’ y ’1’ (sin espacios, comas ni saltos de línea). Describe e implementa un algoritmo, similar al del ejercicio 2, que decodifique este archivo binario e imprima los datos del ejemplar original.

Solución

El esquema de codificación diseñado en el ejercicio 2 fue construido bajo el principio de **codificación en binario puro con longitud fija de 16 bits** , omitiendo intencionalmente

cualquier tipo de carácter separador (como comas, espacios o saltos de línea). Por lo tanto, satisface de manera directa y estricta el requerimiento del protocolo de red móvil.

Referencias Bibliográficas

Referencias

[1] Python Software Foundation. *heapq — Heap queue algorithm*. Python Documentation, 2026. Disponible en: `https://docs.python.org/3/library/heapq.html`.